Binary Tree Right Side View

Difficulty: Medium
Patterns: Tree, Binary Tree, Breadth-First Search, Depth-First Search
LeetCode: https://leetcode.com/problems/binary-tree-right-side-view/

1. Problem Summary

Given the root of a binary tree, return the values visible when looking at the tree from its right side. The result must be ordered from top to bottom.

For `root = [1,2,3,null,5,null,4]`, the right side view is `[1,3,4]`.

At each depth, exactly one node can be visible: the node that appears farthest to the right at that level.

2. Constraints and Observations

- The number of nodes is in the range `[0, 100]`.
- Each value is in the range `[-100, 100]`.
- The tree can be empty, so `root = None` must return `[]`.
- The small node limit means either BFS or DFS is accepted.
- The output contains at most one value per depth, not every node on a right edge.

The important observation is that "right side" means "rightmost by level". A deeper node in a left subtree can still be visible if there is no node to its right at that same depth.

3. Pattern Recognition

This is a binary tree traversal problem. The state we care about is the current depth.

- **Level-order BFS:** process each level and keep the last node.
- **Right-first DFS:** visit `node.right` before `node.left`; the first node reached at each depth is visible.

The DFS version is a clean interview solution because it directly matches the idea: from the right side, try the right child first, and only use the left child when no right-side node exists at that depth.

4. Naive Approach

A first thought is to collect every node by depth, then choose the last node from each depth list.

1. Traverse the whole tree.
2. Store a list of values for every depth.
3. Return the last value from each depth list.

This works, but it stores more information than needed. For each depth, we only need one value: the currently visible candidate.

Time complexity is `O(n)`. Space complexity is also `O(n)` because all node values may be stored across the level lists.

5. Key Intuition

At any depth, the answer needs exactly one node.

If we traverse right child before left child, then the first node we visit at a depth must be the rightmost reachable node at that depth. Once that depth is already recorded, later nodes at the same depth are hidden behind it from the right-side view.

> After right-first DFS has processed any prefix of the traversal, `answer[d]` stores the first node visited at depth `d`, which is the rightmost visible node among the explored nodes at that depth.

6. Optimal Solutions Ranked

Variant 1: BFS Level Order, Take Last Node

- **Idea:** Use a queue. For each level, process all nodes and append the last node's value.
- **Time:** `O(n)`.
- **Space:** `O(w)`, where `w` is the maximum width of the tree.

- **Tradeoff:** Very intuitive and iterative, but the queue can hold many nodes at once.

Variant 2: DFS Left First, Overwrite by Depth

- **Idea:** Traverse left before right. Every time a node is seen at a depth, overwrite that depth's value. The final value at each depth is the rightmost one.
- **Time:** `O(n)`.
- **Space:** `O(h)` recursion stack plus `O(h)` answer.
- **Tradeoff:** Correct, but the overwrite explanation is less direct.

Variant 3: DFS Right First, Record First Node per Depth

- **Idea:** Traverse right before left. When visiting a depth for the first time, append that node's value.
- **Time:** `O(n)`.
- **Space:** `O(h)` recursion stack plus `O(h)` answer.
- **Tradeoff:** Most direct mapping to the right-side view. This is the final recommended solution.

No algorithm can do better than `O(n)` time in the general case because any unseen node might be the only node at a new deepest level and therefore part of the answer.

7. Most Optimal Approach

Use right-first DFS. Maintain an array `view`. During DFS, pass the current `depth`.

- If `depth == len(view)`, this is the first node reached at this depth, so append it.
- Visit the right subtree first.
- Visit the left subtree second.

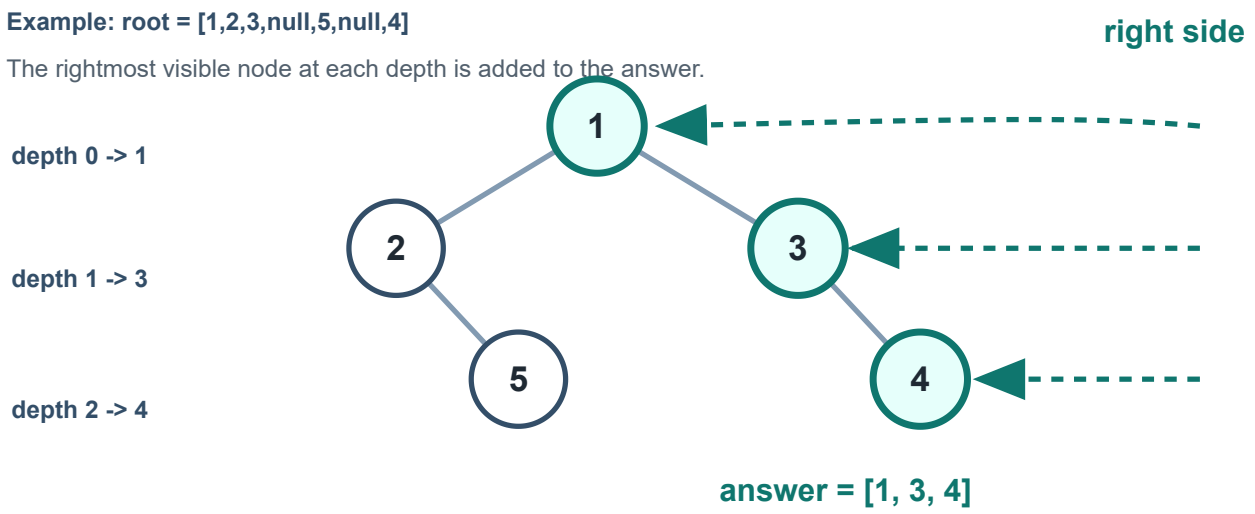
Because the right subtree is visited first, the first node at each depth is exactly the node visible from the right side.

8. Algorithm

1. Create an empty list `view`.
2. Define `dfs(node, depth)`.
3. If `node` is `None`, return.

4. If `depth == len(view)`, append `node.val`.
5. Recursively visit `node.right` with `depth + 1`.
6. Recursively visit `node.left` with `depth + 1`.
7. Call `dfs(root, 0)`.
8. Return `view`.

9. Visual Explanation



From the right side, the visible nodes are the first nodes encountered at depths 0, 1, and 2 when we visit right children before left children.

10. Dry Run

Input: `root = [1,2,3,null,5,null,4]`

```
    1
   /   \
  2     3
   \     \
    5     4
```

Step	Node	Depth	Action	view
1	1	0	First node at depth 0, append 1	`[1]`

Step	Node	Depth	Action	view
2	3	1	First node at depth 1, append 3	`[1, 3]`
3	4	2	First node at depth 2, append 4	`[1, 3, 4]`
4	2	1	Depth already recorded, skip	`[1, 3, 4]`
5	5	2	Depth already recorded, skip	`[1, 3, 4]`

Final answer: `[1, 3, 4]`.

11. Correctness Argument

For any depth `d`, the algorithm visits nodes in right-to-left order relative to that depth. This is because every node's right subtree is fully considered before its left subtree.

When the algorithm first reaches depth `d`, it appends that node's value to `view`. Since nodes at depth `d` are reached from right to left, this first node is the rightmost node at that depth. That is exactly the node visible from the right side.

After a depth is recorded, the algorithm never changes `view[d]`. Any later node at the same depth is farther left, so it cannot be visible from the right side.

The algorithm visits every reachable node, so every non-empty depth is eventually considered. Therefore, `view` contains one value for every visible depth, ordered from top to bottom.

12. Complexity Analysis

- **Time:** `O(n)`, where `n` is the number of nodes. Each node is visited once.
- **Space:** `O(h)` auxiliary recursion stack, where `h` is the height of the tree. The output list stores up to `h` values. In the worst case of a skewed tree, this is `O(n)`.

13. Edge Cases

- Empty tree: return `[]`.
- Single node: return `[root.val]`.
- Only left children: every node is visible because each depth has only one node.

- Only right children: every node is visible.
- A deeper visible node in a left subtree: include it if no node exists to its right at that depth.
- Duplicate values: values do not affect the traversal logic.

14. Final Clean Code

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        view = []

        def dfs(node: Optional[TreeNode], depth: int) -> None:
            if not node:
                return

            if depth == len(view):
                view.append(node.val)

            dfs(node.right, depth + 1)
            dfs(node.left, depth + 1)

        dfs(root, 0)
        return view
```

15. Key Takeaways

- **Main pattern:** tree traversal with depth tracking.
- **Main trick:** visit right before left, then record only the first node seen at each depth.
- **Interview explanation:** "The right side view is one node per level. If I traverse right-first, the first node I encounter at a level is the visible one."
- **Optimality:** `O(n)` time is unavoidable because every node might affect the deepest visible level.